IT4931 / IT5384 / IT4043E · SOICT · HUST

Amazon DynamoDB

Lecture 6 — Highly available key-value store

Consistent hashing · quorum · hinted handoff · Merkle · vector clocks

Learning outcomes

1. **State** DynamoDB's design goals: always-writable, A over C, trusted admin domain
2. **Explain** consistent hashing + virtual nodes for incremental scale
3. **Apply** quorum parameters **N, R, W** and the rule **R + W > N**
4. **Describe** sloppy quorum / hinted handoff and Merkle anti-entropy
5. **Use** vector clocks to reason about concurrent versions

Roadmap

1. **6.1** Place & design considerations
2. **6.2** Partitioning — ring & virtual nodes
3. **6.3** Replication & quorum
4. **6.4** Failures, sync & versioning
5. **6.5** Summary · DynamoDB on AWS today

6.1 Place & design

Two views of Amazon DynamoDB

	System design (this lecture)	DynamoDB on AWS today
What	Distributed KV **architecture** Amazon published & taught	Fully **managed** cloud service
Focus	Hash ring, N/R/W, handoff, Merkle, vector clocks	Tables, keys, capacity, multi-AZ — ops hidden
You learn	Why availability-first KV works at scale	How the **service** exposes those ideas

*Core of today = DynamoDB **architecture**. Then: how that maps to the **AWS service**.*

What DynamoDB is

- **Simple interface** — key/value store (`get` / `put`)
- **Sacrifice strong consistency for availability**
- **Always-writable** — no updates rejected due to failures or concurrent writes
- **Conflict resolution on read**, not on write
- Infrastructure in a **single administrative domain** — nodes are trusted

Why not just a relational DB?

Many Amazon services only need **primary-key** access (cart, session, preferences, catalog lookups).

RDBMS cost	DynamoDB-style store
Complex query / schema you don't use	Simple `get` / `put` on a key
Replication often prefers **consistency**	Tune for **availability** of writes
Hard to scale-out for this pattern	Incremental scale on commodity hosts

Objects are typically small (often < 1 MB). No multi-key transactions.

Motivating example — shopping cart

- Customer must **always** add / remove items — even if disks fail or networks flap
- Rejecting a write = poor customer experience → **always-writable**
- If the latest cart is unavailable, changes still go onto an **older version**
- Later: **merge** divergent carts (app-assisted reconciliation) — an add is never lost
  (deleted items can resurface — trade-off of merge)

SLAs matter — 99.9th percentile

- A storefront page may call **100+** services; each must finish inside a tight budget
- Amazon measures latency at the **99.9th percentile**, not only mean / median
  → every customer should have a good experience, not just the average
- Example SLA shape: respond within ~**300 ms** for **99.9%** of requests at a stated peak load
- Storage is often the critical path → N / R / W and coordinator choice exist to hit that tail

Design considerations

Principle	Meaning
Incremental scalability	Add capacity without redesign / big rewrite
Symmetry	Every node has the **same** responsibilities as peers
Decentralization	Avoid centralized control (historical outage cause)
Heterogeneity	Add stronger nodes without upgrading the whole fleet

System architecture (agenda)

1. **Partitioning**
2. **High availability for writes**
3. **Handling temporary failures**
4. **Recovering from permanent failures**
5. **Membership and failure detection**

Each block has a concrete technique (hash ring, quorum, handoff, Merkle, gossip) — next sections unpack them.

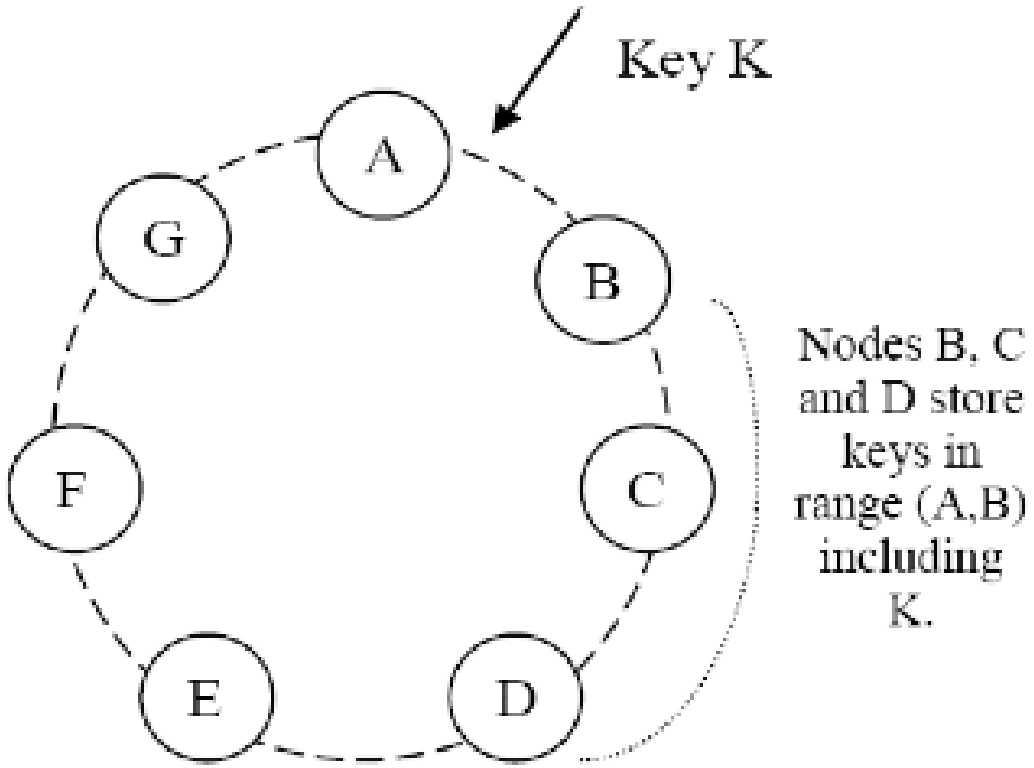
6.2 Partitioning

Consistent hashing

- Hash output treated as a fixed circular space — a **ring**
- Keys and nodes map onto the same ring
- DynamoDB as a **zero-hop DHT** — coordinator can find the owner without multi-hop routing

> **Grand challenge:** every node must maintain an **up-to-date view of the ring**. How?

The ring (partition + replica placement)



Read the figure

- Key **K** hashes into range **(A, B]** → coordinator **B**
- Preference list for K: **B**, **C**, **D** (N = 3 clockwise successors)
- Adding / removing a node moves only **nearby** keys — incremental scale

Virtual nodes

- One **physical** node owns many **virtual** nodes on the ring
- Stronger machines → more virtual nodes (heterogeneity)
- Virtual nodes **spread** around the ring

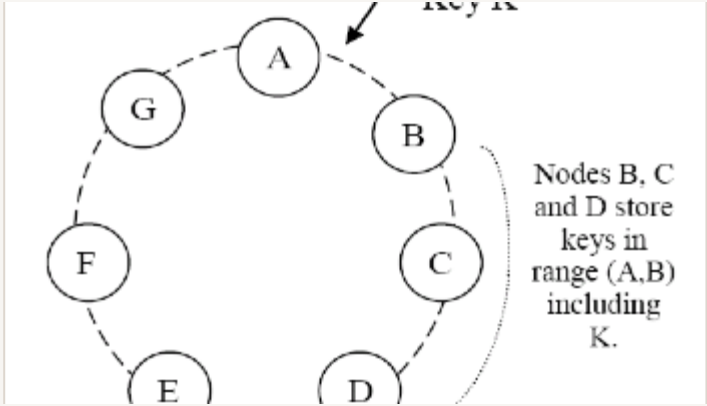
Why it helps

- Node down → its load spreads **evenly** across remaining nodes
- Node joins → takes roughly equal load from each peer
- Capacity of a host controls how many virtual nodes it runs

6.3 Replication & quorum

Replication — preference list

- Each data item is replicated at **N** hosts
- **N** is the size of the **preference list** — nodes responsible for a key
- Same ring figure: B, C, D store keys in range (A, B) including K
- Preference list is built from **distinct physical** nodes (skip duplicate virtual-node hits)
- List is longer than N so failures can fall through to lower-ranked healthy nodes
- In production, replicas are placed across **multiple data centers** when possible

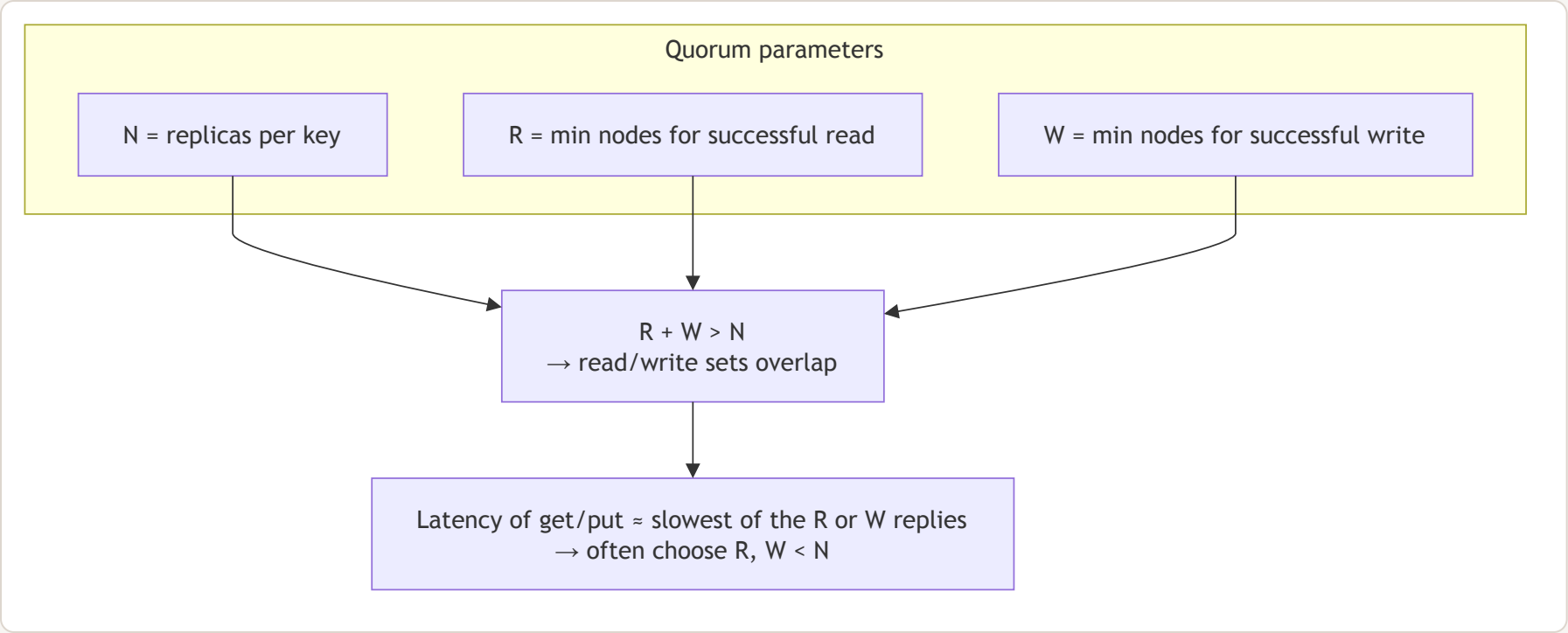


Quorum parameters

Symbol	Meaning
N	Total replicas per key/value
R	Min nodes that must reply for a successful **read**
W	Min nodes that must reply for a successful **write**

Quorum-like system: **R + W > N** → read set and write set **overlap**.

Quorum & latency



Tuning N, R, W

- Latency of `get` / `put` ≈ **slowest** of the R (or W) replicas that must reply
- Therefore R and W are usually **< N** for better latency
- Common production-style setting in the paper: **(N, R, W) = (3, 2, 2)**
- Extremes: **W = 1** maximizes write availability (weaker durability window); higher W/R for stronger overlap

get / put and the coordinator

API	Role
`get(key)`	Fetch replica(s); may return **several versions** + opaque **context**
`put(key, context, object)`	Write a new version; context carries vector-clock metadata from the prior read

- Any node may receive a request; the **coordinator** is typically one of the top N in the preference list
- To protect the **99.9th** latency, writes are often coordinated by a fast node among the top N (e.g. the one that answered the previous read quickly)
- After a `get` sees siblings, the client reconciles and `put` s back — coordinator also **read-repairs** when it can

6.4 Failures, sync & versions

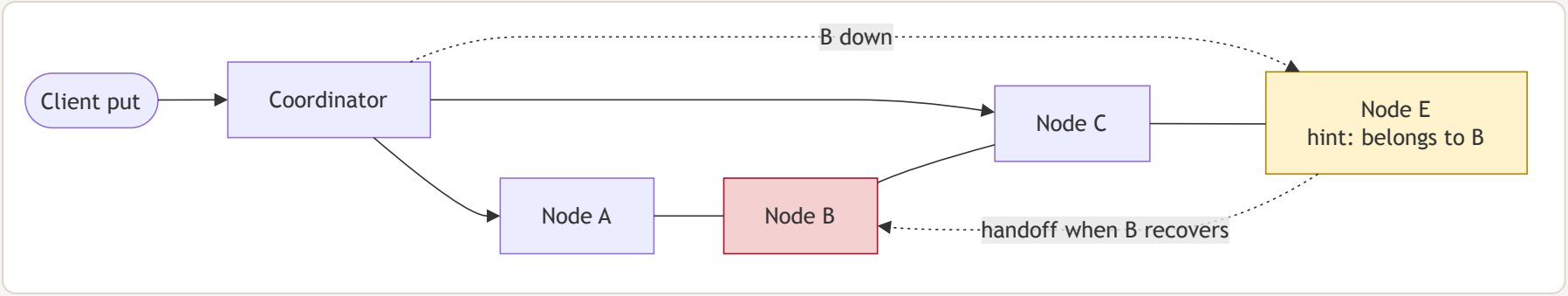
Temporary failures: sloppy quorum

Assume **N = 3**. If **B** is down during a write:

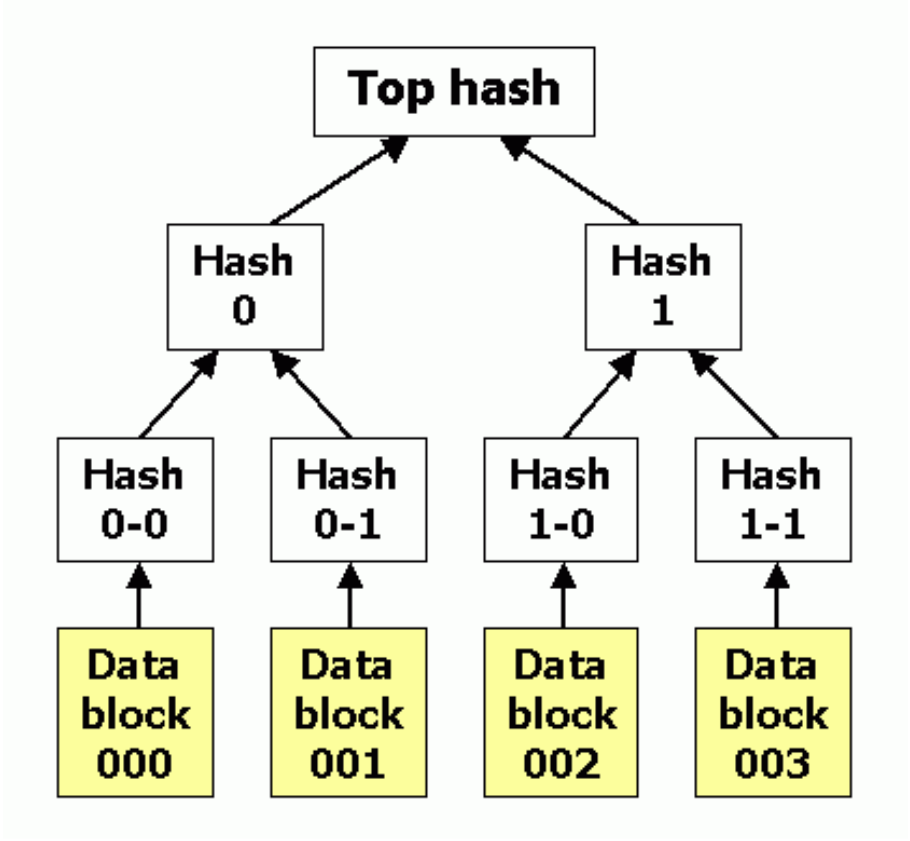
1. Still write **N** reachable nodes (sloppy quorum) — e.g. send replica to **E**
2. **E** is **hinted** that the replica belongs to **B**
3. When **B** recovers, **E** delivers the hint (**hinted handoff**)

Again: **always writeable**.

Hinted handoff



Replica sync — Merkle tree



Why Merkle trees

- Leaves = hashes of individual keys / blocks; parents = hashes of children
- Compare **branches independently** — no need to download the whole tree
- **Anti-entropy**: find divergent replicas and transfer only differing subtrees

Data versioning

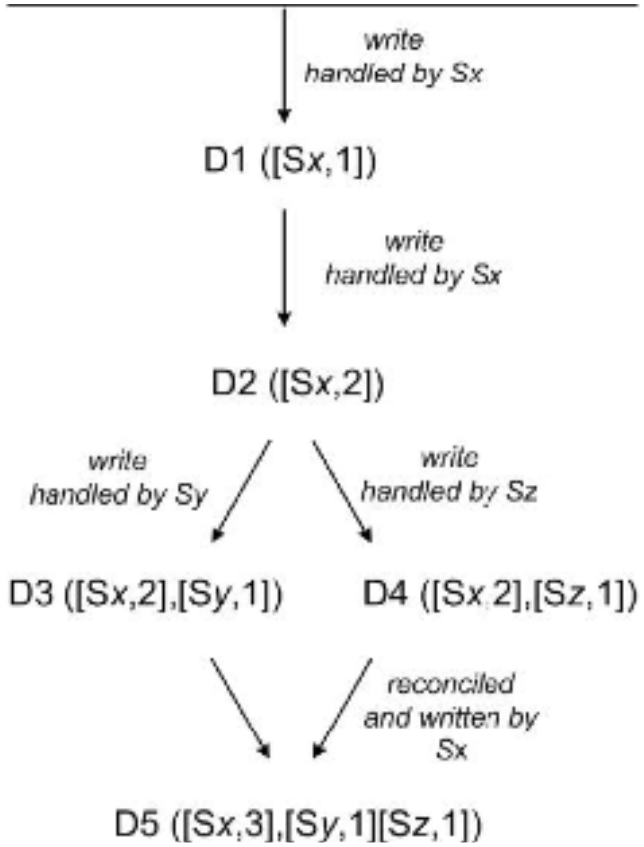
- `put()` may return **before** the update is on all replicas
- `get()` may return **many versions** of the same object
- **Syntactic** reconciliation: system drops ancestors when clocks show causality
- **Semantic** reconciliation: app merges siblings (e.g. shopping-cart union) — or falls back to **last-write-wins**

Vector clock

- A vector clock = list of **(node, counter)** pairs
- Every version of every object has one vector clock
- If all counters of $clock_1 \le clock_2$ → $clock_1$ is an **ancestor** (can be forgotten)
- Concurrent updates → **siblings**; client / app reconciles on read

> When the number of pairs grows past a threshold (e.g. 10), the **oldest** pair is truncated.

Vector clock example



Walkthrough D1 → D5

1. **D1** `[Sx,1]` → **D2** `[Sx,2]` (same writer)
2. Concurrent: **D3** adds Sy · **D4** adds Sz — neither descends from the other
3. **D5** reconciles: max of each component + new write by Sx → `[Sx,3],[Sy,1],[Sz,1]`

Conflicts are detected, not silently overwritten on write.

Membership & failure detection

- **Gossip-based** membership — no centralized registry for who is alive
- Failure detection spreads through the same symmetric gossip plane
- Preserves **symmetry** and **decentralization** from the design principles

6.5 Summary · DynamoDB on AWS

Technical summary

Problem	Technique	Advantage
Partitioning	Consistent hashing	Incremental scalability
HA for writes	Vector clocks + reconcile on read	Version size ≠ update rate
Temporary failures	Sloppy quorum + hinted handoff	Always-writable / durable when some replicas down
Permanent failures	Anti-entropy (Merkle)	Sync divergent replicas in background
Membership	Gossip + failure detection	No central liveness registry

DynamoDB sum-up

- Highly available, scalable store for Amazon.com's e-commerce platform
- Handles server, datacenter, and network-partition failures
- **Incrementally** scalable with request load
- Owners **customize** via **N, R, W**

DynamoDB on AWS today

Fully **managed** NoSQL on AWS — same *ideas* as the Dynamo paper, different *ops surface*:

You design	AWS runs
Partition (+ sort) keys	Hash ring placement & replicas
Access patterns / GSIs	Multi-AZ durability
Eventual vs strong reads	Capacity modes · on-demand / provisioned
Conditional writes · Streams	No handoff / Merkle jobs for you

Hooks: **S3** export, **EMR/Spark**, Lambda triggers — still a **KV/document** store, not a warehouse.

Paper ideas → cloud service

Architecture (this lecture)	DynamoDB on AWS (what you see)
Hash ring + preference list	**Partition key** (and optional sort key) — AWS places replicas
Tune **N, R, W**	Consistency choice per read: **eventual** vs **strong**; capacity modes
Always-writable / HA writes	Multi-**AZ** replication inside a Region — you don't run handoff
Conflict / versions	App-level design + conditional writes; Streams for change feed
You operate nodes	**Serverless / managed** — no ring, gossip, or Merkle jobs for you

Why still learn the architecture?

1. Explains **why** partition keys and hot partitions matter in the cloud product
2. Same HA ideas appear in Cassandra, Riak, and other DynamoDB-inspired stores
3. CAP trade-offs from **L05** become concrete: availability-first KV at scale
4. When you export to the **lake** (S3 / EMR / Spark), you still reason about keys & consistency

Course links

- **L05 NoSQL** — CAP / BASE / KV family (where DynamoDB sits)
- **L07 Presto** — SQL federation over many stores (incl. cloud tables)
- **L08 Kafka** — log + streams (different HA story: ISR / offsets)
- **L04 Lakehouse** — analytics path; DynamoDB often feeds events / export to S3

Takeaways

1. DynamoDB chooses **availability & always-writable** over strong write-side consistency
2. **Consistent hashing + vnodes** enable incremental, heterogeneous scale
3. **N / R / W** are the dials; **R + W > N** for quorum overlap
4. **Hinted handoff** + **Merkle** keep the cluster writable and repaired
5. **App-assisted** conflict resolution + **(3,2,2)**-style quorum keep availability and SLAs practical
6. On AWS, DynamoDB **hides the ring** — but partition keys & consistency still matter

Next

L07 — Presto / Trino